

TRACE-X MASTER IMPLEMENTATION PROMPT

Trust-aware Risk Analysis and Connection
Evidence for Cryptocurrency Exchanges
Smart India Hackathon 2026 prototype

Usage: paste this entire document as the first message (or as the project instruction file, for example CLAUDE.md or AGENTS.md) in your coding assistant. Then instruct it to begin with Phase 1.

PART 1. ROLE AND OPERATING RULES

1.1 Role

You are a principal full-stack engineer with deep experience in blockchain analytics, graph algorithms, forensic-grade evidence handling, and public-sector software. You are building TRACE-X, an evidence-led investigation prototype. You write

production-quality code: typed, tested, documented, and reproducible.

1.2 Working method

- Work in the phases defined in Part 26. Do not start a phase until the previous phase meets its acceptance criteria.
- Before each phase, print a short plan: files to create or change, assumptions, and risks.
- After each phase, run the full test suite, linters, and type checks, and report the results in plain text.
- Prefer small, reviewable commits. Use conventional commit messages (feat, fix, docs, test, refactor, chore).
- When a requirement is ambiguous, choose the most conservative interpretation, record the decision in docs/decisions.md, and continue. Ask the user only when a decision blocks progress.
- Never invent data. Never fabricate exchange addresses, transaction hashes, or citations. If real data is unavailable, use clearly marked synthetic fixtures.
- Never leave placeholder code such as "TODO implement" in delivered modules. If something

is deferred, document it in docs/limitations.md and raise an explicit `NotImplementedError` with a clear message.

1.3 Code quality bar

- Python: 3.11, full type hints, ruff for lint and format, mypy in strict mode for app code, pytest with coverage above 85 percent for backend logic modules.
- TypeScript: strict mode, ESLint plus Prettier, no use of "any" without a justification comment, Vitest and Testing Library for components.
- Functions do one thing. Keep functions under 50 lines where practical. Keep modules cohesive.
- Docstrings on all public functions and classes describing purpose, parameters, return values, and failure modes.
- No dead code, no commented-out code, no debug prints. Use structured logging.
- All magic numbers live in configuration with documented defaults.

1.4 Output style rules for everything you generate (code comments, docs, UI copy, commit messages, README)

- Plain, professional, neutral English. Sentence case.
 - Do not use emojis. Do not use emoji-style icons. Do not use decorative Unicode symbols, box-drawing decoration, ornamental bullets, or typed arrow and checkmark characters.
 - Do not use exclamation marks, hype words, or marketing filler.
 - Use ASCII hyphens and standard punctuation only.
 - In documentation, use standard headings, numbered lists, and tables. Keep tone similar to a technical standards document.
-

PART 2. PRODUCT CONTEXT AND SCOPE

2.1 Problem statement

Real-time identification of cryptocurrency exchanges potentially connected to fraud, starting from suspect wallet addresses reported by victims, using automated blockchain analytics.

2.2 Core concept

A victim-report-conditioned, uncertainty-aware

blockchain investigation system that:

1. Traces reported funds through a bounded, case-focused transaction graph.
2. Assesses potential exchange or service connections with explicit attribution confidence and provenance.
3. Corroborates related reports through shared on-chain evidence, with temporal updating.

2.3 Primary user journey

1. The user opens a single-screen page.
2. The user pastes a wallet address. Optional inputs: transaction hash, incident date, reported amount, chain.
3. The user selects "Check Wallet".
4. The system validates input, creates a case, retrieves transactions, builds a bounded graph, extracts paths, applies attribution, computes behavioral signals, and checks related reports.
5. The user sees a plain-language result with expandable evidence, and can export an evidence report as PDF.

2.4 Users

- Victims and members of the public with no blockchain expertise.
- Cybercrime help desk staff who need a structured lead.
- Investigators and compliance analysts who need traceable evidence.

2.5 In scope for the prototype

- One primary chain: Ethereum mainnet, native ETH and selected ERC-20 tokens (USDT, USDC, DAI).
- Adapter interface designed so additional chains can be added without changing the graph, attribution, or report layers.
- Bounded graph expansion with configurable limits.
- Provenance-tracked label registry for exchanges and services.
- Interpretable behavioral signals with a rule baseline and a scikit-learn baseline.
- Cross-report corroboration demonstrated on synthetic or curated cases.
- Evidence-linked PDF report and case audit trail.

- Evaluation scripts with baselines and ablations.

2.6 Out of scope and must not be claimed

- Declaring any person, wallet owner, or exchange criminal based on a model output.
- Guaranteed tracing across mixers, bridges, privacy systems, or unsupported chains.
- Replacing law enforcement, exchange compliance teams, or legal process.
- Asserting that an address belongs to an exchange without a cited source or a clearly marked inference.
- Graph neural networks in the MVP.
- Multi-chain support in the MVP.

PART 3. NON-NEGOTIABLE PRINCIPLES

These are hard constraints. Encode them in code, tests, and copy. If any instruction elsewhere conflicts with these, these win.

P1. Observed fact versus inference.

Every data point in an output must be tagged as

one of: observed (directly from chain data), derived (computed deterministically from observed data), or inferred (heuristic or model-based). The API schema must carry this tag. The UI must visually and textually distinguish them.

P2. No single opaque score.

Do not produce a single "fraud score" or "risk percentage". Provide separate fields: path evidence, attribution confidence, behavioral signals, corroboration strength. An internal prioritization value may exist only for sorting paths, must be documented, and must never be shown to end users as a probability of wrongdoing.

P3. Abstention is a first-class outcome.

The system must be able to return "Insufficient data" and "No match found in current references". These are valid, expected results and must be covered by tests and UI states.

P4. Exchange link is not exchange complicity.

Never describe an exchange as fraudulent, criminal, complicit, or guilty. Approved phrasing is defined in the copy deck (Part 20).

P5. Shared nodes are not common ownership.

Overlap between cases on an exchange deposit

address, a service contract, a bridge, a mixer, or a high-degree hub must never be presented as evidence of common control. Flag it as a possible shared-infrastructure explanation.

P6. Full path traceability.

Every surfaced path retains, for each edge: transaction hash, block number, timestamp (UTC), sender, receiver, asset identifier (native or contract address), raw amount, decimal-adjusted amount, and data source.

P7. Provenance on every label.

Every entity label stores: entity name, address, chain, source name, source type, source URL or reference, date observed, date last verified, scope, verification level, and notes.

P8. Reproducibility.

Given the same input and the same data snapshot, the system must produce identical structured output. Store the parameters, data source identifiers, retrieval times, code version, and model version with each case.

P9. Minimal data collection.

Collect only what is needed. Victim narrative and contact details are optional, stored separately,

access-controlled, and never mixed into public-chain evidence tables.

P10. Honest limits.

Every result includes a limitations block listing missing data, unsupported transitions, label freshness, and analysis timestamp.

PART 4. CONTROLLED VOCABULARY

Use these terms consistently across code identifiers, database enums, API fields, UI, and documentation.

4.1 Entity types (enum entity_type)

- exchange
- exchange_hot_wallet
- exchange_deposit_address
- payment_service
- mixer
- bridge
- smart_contract_service
- decentralized_exchange
- other_service

- unknown

4.2 Connection types (enum connection_type)

- direct: the reported address transfers directly to a labelled address.
- indirect: a path with one or more intermediate addresses reaches a labelled address.
- inferred: association depends on a heuristic clustering or similarity inference, not a labelled address in the observed path.

4.3 Attribution states (enum attribution_state)

- verified_label_match
- potential_association
- no_match_in_current_references
- insufficient_data

4.4 Verification levels (enum verification_level)

- level_3_primary_source: published by the entity itself (for example official proof of reserves or documentation).
- level_2_reputable_secondary: published by a reputable public source with method described.

- level_1_community_or_unverified: community-sourced or single-source, unverified.
- level_0_synthetic: fabricated for testing. Must never appear in non-demo mode.

4.5 Evidence tags (enum evidence_tag)

- observed
- derived
- inferred

4.6 Path break reasons (enum path_break_reason)

- mixer_interaction
- bridge_interaction
- unsupported_contract
- swap_asset_change
- privacy_protocol
- expansion_limit_reached
- data_unavailable
- time_window_exceeded

4.7 Corroboration strength (enum corroboration_strength)

- none
- weak: single shared address that is high-degree or labelled shared infrastructure.

- moderate: shared non-infrastructure address or overlapping path segment of length one.
- strong: overlapping path segment of length two or more, with consistent temporal ordering, excluding shared infrastructure.

4.8 Behavioral signal severity (enum signal_level)

- not_observed
- observed_low
- observed_moderate
- observed_high

Signals describe how strongly a pattern is present, not the likelihood of fraud.

PART 5. TECHNOLOGY STACK

5.1 Backend

- Python 3.11
- FastAPI, Uvicorn, Pydantic v2, pydantic-settings
- SQLAlchemy 2.x with Alembic migrations
- PostgreSQL 15 or later
- httpx for outbound HTTP with timeouts and retries (tenacity for backoff)

- NetworkX for the graph prototype
- scikit-learn, pandas, numpy for the behavioral baseline
- joblib for model persistence
- ReportLab for PDF generation
- structlog for structured logging
- slowapi (or equivalent) for rate limiting
- eth-utils for address checksum validation
- pytest, pytest-asyncio, pytest-cov, respx (HTTP mocking), hypothesis (property tests)
- ruff, mypy

5.2 Frontend

- React 18, Vite, TypeScript (strict)
- Tailwind CSS with design tokens defined in Part 19
- TanStack Query for data fetching
- React Router
- React Flow for the path graph view
- Lucide React for icons (functional use only)
- Vitest, Testing Library, Playwright for one end-to-end smoke test
- ESLint, Prettier

5.3 Infrastructure

- Docker and docker-compose for local development (services: api, db, frontend)
- Makefile targets: setup, dev, test, lint, typecheck, migrate, seed, eval, build, clean
- GitHub Actions workflow for lint, type check, tests on pull requests

5.4 Chain data

- Primary adapter: Etherscan-compatible explorer API (account transaction list, internal transaction list, ERC-20 token transfer list, transaction receipt status, block by timestamp).
- Secondary adapter: JSON-RPC node adapter interface stub with documented gaps.
- Mock adapter: fixture-driven adapter used for tests and for offline demo mode. Selected via configuration.

PART 6. REPOSITORY STRUCTURE

Create exactly this structure. Add files only where they serve a stated purpose.

trace-x/
README.md
LICENSE
Makefile
docker-compose.yml
.env.example
.gitignore
.editorconfig
.github/workflows/ci.yml
docs/
architecture.md
api.md
data-model.md
attribution-method.md
graph-method.md
behavioral-signals.md
corroboration-method.md
evaluation.md
limitations.md
security-and-privacy.md
decisions.md
demo-script.md
copy-deck.md
backend/
pyproject.toml
alembic.ini

alembic/versions/

app/

main.py

api/

deps.py

errors.py

v1/

router.py

cases.py

labels.py

health.py

core/

config.py

logging.py

security.py

rate_limit.py

constants.py

enums.py

time.py

adapters/

base.py

etherscan.py

rpc_stub.py

mock.py

cache.py

registry.py

models/
base.py
case.py
transaction.py
label.py
evidence.py
audit.py
report_link.py
schemas/
case.py
transaction.py
graph.py
path.py
attribution.py
behavior.py
corroboration.py
evidence.py
common.py
services/
case_service.py
ingestion_service.py
graph_service.py
attribution_service.py
behavior_service.py
corroboration_service.py
evidence_service.py

report_service.py
graph/
builder.py
expansion.py
paths.py
value_tracing.py
breaks.py
ranking.py
attribution/
registry.py
matcher.py
confidence.py
infrastructure.py
behavior/
features.py
rules.py
baseline_model.py
explain.py
corroboration/
subgraph.py
matcher.py
thresholds.py
evidence/
assembler.py
audit.py
limitations.py

reports/
pdf.py
templates.py
styles.py
evaluation/
datasets.py
splits.py
metrics.py
baselines.py
ablations.py
run_all.py
data/
fixtures/
label_registry/
models/
tests/
unit/
integration/
property/
fixtures/
frontend/
package.json
tsconfig.json
vite.config.ts
tailwind.config.ts
postcss.config.js

index.html
src/
main.tsx
App.tsx
styles/
tokens.css
globals.css
lib/
api.ts
types.ts
format.ts
copy.ts
validation.ts
pages/
HomePage.tsx
ResultPage.tsx
NotFoundPage.tsx
components/
layout/
form/
result/
evidence/
graph/
feedback/
common/
tests/

PART 7. CONFIGURATION AND ENVIRONMENT

7.1 Environment variables (document each in .env.example with a comment and safe default)

Application

- APP_ENV: development, test, production.
Default development.
- APP_VERSION: injected from build metadata.
- LOG_LEVEL: default INFO.
- CORS_ALLOWED_ORIGINS: comma separated list. No wildcard in production.
- DEMO_MODE: true or false. When true, level_0_synthetic labels and mock adapter are permitted and every response and report carries a visible demonstration notice.

Database

- DATABASE_URL: postgresql+psycopg URL.

Chain data

- CHAIN_DEFAULT: ethereum.
- DATA_ADAPTER: etherscan, rpc, or mock.

- EXPLORER_API_BASE_URL
- EXPLORER_API_KEY (server side only, never logged, never returned)
- EXPLORER_TIMEOUT_SECONDS: default 15.
- EXPLORER_MAX_RETRIES: default 3.
- EXPLORER_RATE_LIMIT_PER_SECOND: default 4.
- CACHE_TTL_SECONDS: default 3600 for historical data, 60 for recent data.

Graph limits (defaults, all overridable per request within hard maximums)

- GRAPH_MAX_HOPS_DEFAULT: 4. Hard maximum 6.
- GRAPH_MAX_NODES_DEFAULT: 500. Hard maximum 2000.
- GRAPH_MAX_EDGES_DEFAULT: 2000. Hard maximum 8000.
- GRAPH_TIME_WINDOW_DAYS_DEFAULT: 30. Hard maximum 180.
- GRAPH_HIGH_DEGREE_THRESHOLD: 200. Nodes above this out-degree or in-degree are treated as hubs and not expanded.
- GRAPH_MIN_TRANSFER_VALUE_USD_EQUIVALENT: 0 by default. Dust filtering is optional and must

be disclosed in the report when used.

- GRAPH_VALUE_TRACING_METHOD:
proportional or fifo. Default proportional.

Attribution

- LABEL_STALE_AFTER_DAYS: default 365.
Labels older than this are flagged as stale.
- ALLOW_SYNTHETIC_LABELS: default false,
forced false when DEMO_MODE is false.

Corroboration

- CORROBORATION_MIN_SHARED_PATH_LENGTH:
default 2.
- CORROBORATION_MAX_TIME_SKEW_DAYS:
default 45.
- CORROBORATION_HUB_EXCLUSION_DEGREE:
default 100.

API protection

- RATE_LIMIT_CASES_PER_MINUTE_PER_IP:
default 10.
- REQUEST_MAX_BODY_BYTES: default 16384.

Reports

- REPORT_RETENTION_DAYS: default 90.
- CASE_RETENTION_DAYS: default 180.

7.2 Configuration rules

- Load configuration through a single typed settings object. Fail fast at startup if required variables are missing for the selected mode.
 - Never read environment variables outside `core/config.py`.
 - Secrets are never included in logs, error messages, API responses, or PDF reports.
 - Provide a redacted configuration summary endpoint only in development mode.
-

PART 8. DATABASE SCHEMA

Use SQLAlchemy models and Alembic migrations. All timestamps are timezone-aware UTC. All primary keys are UUID unless stated. Add `created_at` and `updated_at` to every table. Add indexes as listed.

8.1 Table: cases

- `id` (uuid, pk)
- `case_reference` (text, unique, format TX-YYYY-NNNNNN)
- `chain` (text, not null)

- reported_address (text, not null, stored in canonical form)
 - reported_tx_hash (text, nullable)
 - incident_date (timestampz, nullable)
 - incident_date_precision (enum: exact, day, approximate, unknown)
 - reported_amount (numeric, nullable)
 - reported_asset (text, nullable)
 - status (enum: pending, running, completed, partial, failed)
 - parameters (jsonb, not null): the effective graph and analysis parameters used
 - data_snapshot_id (text): identifier for the data retrieval set used
 - code_version (text)
 - model_version (text, nullable)
 - is_demo (boolean, not null)
 - analysis_started_at, analysis_completed_at (timestampz)
 - failure_reason (text, nullable)
- Indexes: unique(case_reference), index(chain, reported_address), index(created_at).

8.2 Table: case_private_details

Separate from public-chain evidence. Access

controlled.

- case_id (uuid, pk, fk cases.id)
- narrative (text, nullable, encrypted at rest at application level)
- contact_reference (text, nullable, encrypted at rest at application level)
- consent_recorded (boolean, not null)
- retention_until (timestampz)

8.3 Table: transactions

Public chain facts only.

- id (uuid, pk)
- chain (text)
- tx_hash (text)
- log_index (integer, default 0): distinguishes multiple transfers in one transaction
- transfer_kind (enum: native, internal, token)
- block_number (bigint)
- block_timestamp (timestampz)
- sender (text)
- receiver (text)
- asset_id (text): "native" or the token contract address in canonical form
- asset_symbol (text, nullable)

- asset_decimals (integer, nullable)
 - amount_raw (numeric(78,0))
 - amount_decimal (numeric, nullable)
 - status (enum: success, failed)
 - source (text)
 - retrieved_at (timestamptz)
- Unique constraint: (chain, tx_hash, log_index, transfer_kind, sender, receiver, asset_id).
 Indexes: (chain, sender, block_timestamp), (chain, receiver, block_timestamp), (chain, tx_hash).

8.4 Table: case_transactions

Links a case to the transactions retained in its bounded graph.

- case_id, transaction_id (composite pk)
- hop_depth (integer)
- direction (enum: outgoing, incoming)
- included_reason (text)

8.5 Table: entity_labels

- id (uuid, pk)
- entity_name (text, not null)
- entity_type (enum entity_type)
- chain (text)

- address (text, not null, canonical form)
- cluster_id (uuid, nullable): only for inferred clusters
- label_origin (enum: observed_label, inferred_cluster)
- source_name (text, not null)
- source_type (enum: primary_disclosure, reputable_secondary, community, dataset, synthetic)
- source_reference (text, not null): URL or document identifier
- verification_level (enum verification_level)
- observed_at (date, not null)
- last_verified_at (date, nullable)
- scope_notes (text): for example "hot wallet only" or "deposit addresses not covered"
- is_shared_infrastructure (boolean, default false)
- is_active (boolean, default true)
Unique constraint: (chain, address, source_name).
Indexes: (chain, address), (entity_name).

8.6 Table: label_change_log

- id, label_id, change_type (created, updated, deactivated), changed_at, changed_by, previous_value (jsonb), new_value (jsonb)

8.7 Table: case_paths

- id (uuid, pk)
- case_id (fk)
- path_index (integer): rank order within the case
- hop_count (integer)
- start_address, end_address (text)
- first_timestamp, last_timestamp (timestampz)
- elapsed_seconds (integer)
- traced_value_amount (numeric), traced_value_asset (text)
- tracing_method (enum: proportional, fifo, none)
- value_continuity_ratio (numeric): traced value divided by originating transfer value, 0 to 1
- relevance_criteria (jsonb): the itemized criteria and values used to rank this path
- has_break (boolean)
- break_reason (enum path_break_reason, nullable)

- edges (jsonb): ordered list of transaction references with full P6 fields

8.8 Table: case_attributions

- id (uuid, pk)
- case_id, path_id (fk)
- label_id (fk entity_labels)
- connection_type (enum connection_type)
- attribution_state (enum attribution_state)
- attribution_confidence_level (enum: high, medium, low, none)
- confidence_factors (jsonb): itemized factors, see Part 12
- label_is_stale (boolean)
- shared_infrastructure_flag (boolean)
- explanation (text)

8.9 Table: case_signals

- id, case_id, path_id (nullable)
- signal_key (text): for example rapid_pass_through
- level (enum signal_level)
- evidence_tag (enum evidence_tag)
- feature_values (jsonb)

- explanation (text)
- limitation_note (text)

8.10 Table: case_links (cross-report corroboration)

- id, case_a_id, case_b_id (fk), ordered so that case_a_id is less than case_b_id
- strength (enum corroboration_strength)
- evidence_tag (enum evidence_tag)
- shared_addresses (jsonb)
- shared_transactions (jsonb)
- shared_path_segments (jsonb)
- shared_infrastructure_flag (boolean)
- temporal_notes (text)
- first_observed_at (timestampz)
- last_updated_at (timestampz)
- change_history (jsonb): list of updates with the evidence that changed the link
Unique constraint: (case_a_id, case_b_id).

8.11 Table: audit_events

- id, case_id (nullable), event_type, occurred_at, actor (system or role identifier), payload (jsonb, sanitized), payload_hash (text)
Event types include: case_created,

retrieval_started, retrieval_completed,
retrieval_failed, graph_built, attribution_applied,
signals_computed, corroboration_run,
report_generated, case_deleted.

8.12 Table: reports

- id, case_id, generated_at, file_hash_sha256,
storage_reference, retention_until

8.13 Migration rules

- One migration per logical change with a descriptive name.
- Include downgrade paths.
- Provide a seed command that loads the label registry seed and demo fixtures, refusing to load level_0_synthetic entries unless DEMO_MODE is true.

PART 9. API SCHEMAS (PYDANTIC V2)

Define strict models. Use enums from Part 4.
Forbid extra fields on input models. Every output object that carries a claim includes an evidence_tag.

9.1 CaseCreateRequest

- address: string, required, 1 to 128 characters, validated per chain
- chain: string, default from settings, must be a supported chain
- tx_hash: string, optional, validated format
- incident_date: date or datetime, optional, not in the future
- incident_date_precision: enum, default unknown
- reported_amount: decimal, optional, non-negative
- reported_asset: string, optional
- parameters: optional override object (max_hops, time_window_days, max_nodes, max_edges), each within hard limits
- narrative: string, optional, max 4000 characters, stored in private details
- consent_for_narrative_storage: boolean, required true if narrative present

9.2 CaseSummaryResponse

- case_reference, status, chain, reported_address, analysis_completed_at

- headline_state: one of the attribution states, or "no_paths_found"
- summary_text: plain-language sentence set generated from templates in the copy deck
- confidence_panel: object with four independent entries
 - path_evidence: level (strong, moderate, limited, none) and factors
 - attribution_confidence: level (high, medium, low, none) and factors
 - behavioral_signals: list of signal summaries
 - corroboration_strength: enum
- limitations: list of limitation objects
- demo_notice: string or null

9.3 PathResponse

- path_id, rank, hop_count, start_address, end_address
- edges: list of EdgeRef
- elapsed_seconds, traced_value, tracing_method, value_continuity_ratio
- relevance_criteria: itemized list with name, value, and contribution note

- break: object or null with reason and description
- evidence_tag

9.4 EdgeRef

- tx_hash, log_index, block_number, timestamp_utc, sender, receiver
- asset_id, asset_symbol, amount_raw, amount_decimal
- transfer_kind, source, retrieved_at
- explorer_url (constructed from configuration, not stored as a fact)

9.5 AttributionResponse

- entity_name, entity_type, matched_address
- connection_type, attribution_state
- label: source_name, source_type, source_reference, verification_level, observed_at, last_verified_at, scope_notes, is_stale
- confidence_level and confidence_factors (list of name, value, direction)
- shared_infrastructure_flag
- explanation
- evidence_tag

9.6 SignalResponse

- signal_key, display_name, level, explanation, limitation_note, feature_values, evidence_tag

9.7 RelatedCaseResponse

- related_case_reference
- strength, evidence_tag
- shared_addresses, shared_transactions, shared_path_segments (each item with full P6 fields where applicable)
- shared_infrastructure_flag
- caution_text (mandatory, from copy deck)
- first_observed_at, last_updated_at

9.8 ErrorResponse (uniform)

- error: object with code (machine readable), message (human readable), field (optional), request_id
Error codes: invalid_address, invalid_chain, invalid_tx_hash, invalid_date, limit_exceeded, rate_limited, upstream_unavailable, upstream_timeout, case_not_found, report_not_ready, internal_error.
-

PART 10. DATA ADAPTER LAYER

10.1 Interface (adapters/base.py)

Define an abstract class ChainDataAdapter with these async methods:

- `validate_address(address)` returns canonical address or raises `InvalidAddressError`
- `get_native_transfers(address, start_block, end_block, direction)`
- `get_internal_transfers(address, start_block, end_block)`
- `get_token_transfers(address, start_block, end_block, contract_filter)`
- `get_transaction(tx_hash)`
- `block_for_timestamp(timestamp, closest)`
- `describe()` returns adapter name, version, capabilities, and known gaps

All methods return normalized

NormalizedTransfer objects (schema in 10.2) plus an AdapterResult wrapper containing: items, source_name, retrieved_at, request_count, truncated (boolean), warnings (list), and errors (list).

10.2 NormalizedTransfer

chain, tx_hash, log_index, transfer_kind,
block_number, block_timestamp, sender, receiver,
asset_id, asset_symbol, asset_decimals,
amount_raw, status, source, retrieved_at.

Rules:

- Addresses are canonical (EIP-55 checksum for Ethereum) at the boundary of the adapter.
- Amounts are integers in the smallest unit. Decimal-adjusted amounts are derived using asset_decimals; if decimals are unknown, leave amount_decimal null and add a warning.
- Failed transactions are retained with status failed but excluded from value tracing.
- Contract creation and self-transfers are retained but flagged.

10.3 Etherscan-compatible adapter behavior

- Respect the rate limit with a token bucket.
- Handle pagination fully up to a configurable maximum per address (default 10000 records). If the maximum is reached, set truncated to true and add a warning that history is incomplete. Never silently drop data.

- Retries with exponential backoff and jitter for timeouts and 5xx responses. Do not retry on 4xx other than 429.
- Detect and handle the "No transactions found" response as an empty successful result, not as an error.
- Map upstream error strings to typed exceptions. Do not leak upstream error text containing keys or URLs to clients.
- Record every upstream call in the audit trail with a sanitized request descriptor and response size.

10.4 Caching (adapters/cache.py)

- Cache responses keyed by adapter, method, and normalized parameters.
- Historical ranges (block range fully below the current head minus a finality margin of 64 blocks) use the long TTL.
- Ranges including recent blocks use the short TTL.
- Cache entries store `retrieved_at` so reports can state data freshness.
- Provide cache bypass for evaluation and reproducibility runs.

10.5 Mock adapter

- Reads JSON fixtures from `backend/data/fixtures`.
- Fixture files describe a small chain: addresses, transfers, labels, expected outputs.
- Deterministic. No network access.
- Every response from the mock adapter is marked source "mock_fixture" and triggers the demonstration notice.

10.6 Known adapter gaps to document in `docs/limitations.md`

- Internal transfers depend on explorer tracing coverage.
- Token transfers that do not emit standard events are not captured.
- Contract-mediated swaps change asset identity and are treated as path breaks with reason `swap_asset_change` unless a decoding rule exists.
- Explorer data may be delayed or reorganized near chain head.

PART 11. GRAPH ENGINE

11.1 Data structure

Use a NetworkX MultiDiGraph. Nodes are canonical addresses with attributes: first_seen, last_seen, in_degree_observed, out_degree_observed, is_contract (if known), is_hub, label_ids. Edges are individual transfers keyed by (tx_hash, log_index) with attributes from NormalizedTransfer.

11.2 Case-focused bounded expansion

(graph/expansion.py)

Input: reported address, optional tx hash, incident date with precision, parameters.

Algorithm outline:

1. Determine the seed set.
 - If tx_hash is supplied and resolves to a transfer involving the reported address, seed with that transfer and its receiver, and mark the seed as anchored.
 - Otherwise, seed with the reported address and a time window centered on the incident date (if given), else the most recent window.
2. Expand outward breadth-first by hop up to max_hops. For each frontier address:

- Retrieve outgoing transfers within the allowed time window. The window for hop n starts at the arrival time of the traced value at that hop and extends forward by the configured window.
 - Apply node and edge budgets. When a budget is reached, stop expansion, record an `expansion_limit_reached` break, and set the partial flag.
 - Do not expand hubs (degree above `GRAPH_HIGH_DEGREE_THRESHOLD`) or addresses with an active label of type `exchange`, `exchange_hot_wallet`, `exchange_deposit_address`, `mixer`, `bridge`, or `decentralized_exchange`. Record them as terminal nodes with the reason. Terminal labelled nodes are the points where attribution is applied.
3. Optionally expand incoming transfers one hop for context (configurable, default off) to show funding source of the reported address. Mark these edges direction incoming.
 4. Deduplicate transfers. Keep only successful transfers for value tracing.

5. Persist the retained transfers and case-transaction links.

11.3 Path extraction (graph/paths.py)

- Enumerate simple paths from the seed to each terminal or leaf node within the hop limit, using time-respecting constraints: each successive edge timestamp must be greater than or equal to the previous edge timestamp, and within the per-hop window.
- Prune paths whose value continuity ratio falls below a configurable minimum (default 0.05) and record the pruning in the case parameters.
- Cap the number of returned paths (default 25) and report how many candidates were omitted.

11.4 Value tracing (graph/value_tracing.py)

Implement both methods behind a common interface. Document the method in docs/graph-method.md with worked examples.

- proportional: when an address receives value from multiple sources and sends to multiple destinations, attribute outgoing value to incoming sources in proportion to each

incoming source's share of the address's traced pool at that time.

- fifo: outgoing value consumes earliest received traced value first.

Rules:

- Operate per asset. A change of asset identity ends the traced value with a `swap_asset_change` break unless the swap is explicitly decoded.
- Account for gas fees for native transfers when computing continuity, but do not attribute fees as a loss of traceability.
- Output `traced_value` and `value_continuity_ratio` for each path, tagged derived.
- State plainly in every report that the method estimates value movement and does not prove that specific units of value moved along the path.

11.5 Path breaks (`graph/breaks.py`)

Detect and label breaks using entity labels of type mixer or bridge, a maintained list of known privacy-protocol contract addresses (with sources), interactions with unlabelled contracts that are not simple transfers, and asset changes.

A break ends tracing along that branch and is displayed in the path with its reason.

11.6 Relevance ranking (graph/ranking.py)

Rank paths by a transparent, documented ordered set of criteria. Do not combine into a hidden score. Use a lexicographic or explicit weighted-sum approach where the weights are in configuration and every criterion value is stored in `relevance_criteria` and shown in the evidence view.

Criteria:

1. Anchored to the reported transaction (yes or no).
2. Value continuity ratio.
3. Time proximity of the first edge to the incident date.
4. Hop count (shorter preferred when other criteria tie).
5. Terminal node is labelled (informational ordering, not a claim).

Document that ranking orders the display and does not measure wrongdoing.

11.7 Pattern detection on the graph

Detect and record as derived facts: splitting (out-degree of 3 or more within a defined interval),

merging (in-degree of 3 or more of traced value), rapid pass-through (onward transfer within a configurable time after receipt), repeated equal-value transfers, and peel-chain shape (repeated small outputs with a large remainder forwarded). Each detection stores the transaction references that support it.

11.8 Performance and safety

- Target end-to-end latency under 20 seconds for default limits on the mock adapter and under 60 seconds on a live adapter, excluding upstream rate-limit waits.
- Use asyncio concurrency with a bounded semaphore for adapter calls.
- Every expansion loop checks the budgets and a wall-clock deadline (default 90 seconds). On deadline, return a partial result with a `data_unavailable` or `expansion_limit_reached` limitation.
- Property tests must confirm that no path violates time ordering and that budgets are never exceeded.

PART 12. ENTITY ATTRIBUTION

12.1 Label registry

- Source of truth is the `entity_labels` table, seeded from `backend/data/label_registry/*.json`.
- Seed only with addresses that you can cite from publicly documented sources (for example exchange proof-of-reserves publications or official documentation). Record the URL and observation date for each. If you cannot verify an address, do not include it. Provide a small set of clearly marked `level_0_synthetic` entries for demonstration, loaded only in `DEMO_MODE`.
- Provide an admin-only CLI to add, update, and deactivate labels. Every change writes to `label_change_log`.

12.2 Matching (`attribution/matcher.py`)

- Exact match on canonical address and chain.
- Determine `connection_type`: direct when the labelled address is the first receiver of the seed transfer, indirect when reached after one or more intermediaries, inferred only when derived from a cluster label with `label_origin` `inferred_cluster`.

- If a path terminates at an unlabelled address, produce attribution_state no_match_in_current_references for that path. If retrieval was truncated or the path has a break, produce insufficient_data instead.

12.3 Confidence factors

(attribution/confidence.py)

Do not compute a probability. Produce a level (high, medium, low, none) from an explicit rule table, and return every factor used.

Factors:

- verification_level of the label
 - label_origin (observed_label or inferred_cluster)
 - label age relative to LABEL_STALE_AFTER_DAYS
 - scope_notes match (for example label scope excludes deposit addresses)
 - number of independent sources agreeing on the label
 - connection_type
 - path integrity (breaks or truncation present)
- Rule table (documented in docs/attribution-method.md):

- high: level_3 or two or more independent level_2 sources, observed_label, not stale, direct or indirect, path integrity intact.
- medium: single level_2 source, or level_3 but stale, observed_label, path integrity intact.
- low: level_1 only, inferred_cluster, stale and single source, or path with break.
- none: no label, or synthetic outside demo mode.

Map states: verified_label_match requires high or medium with observed_label and non-synthetic; potential_association covers low or inferred; the remaining two states are as defined in 12.2.

12.4 Shared infrastructure (attribution/infrastructure.py)

- Maintain a list of addresses and contract types known to be shared infrastructure (exchange hot wallets used by many deposit addresses, routers, bridges, mixers, custodial services). Each entry has a source.
- Set shared_infrastructure_flag on attributions and corroboration links that involve these nodes.

12.5 Explanations

Generate explanation text from templates in the copy deck. Explanations must name the label source, the connection type, the verification level, and any staleness or scope caveat. Never state or imply exchange knowledge or involvement.

PART 13. BEHAVIORAL SIGNALS

13.1 Principles

Signals are triage indicators. They describe observed patterns and always carry a limitation note. They are not a verdict and not a probability.

13.2 Feature families (behavior/features.py)

Temporal

- median and minimum seconds between receipt and onward transfer per hop
- burst count: transfers within a rolling window
- activity concentration: share of activity within 24 hours of incident

Structural

- in-degree and out-degree of path nodes
- split factor and merge factor

- path depth and branching ratio
Value movement
- number of distinct amounts, repeated amount count
- ratio of onward value to received value per hop
- asset changes
- round-number proportion
Counterparty
- count of distinct counterparties per path node
- count and type of labelled counterparties
- interaction with hub nodes
Case context
- time offset between incident date and first traced transfer
- proximity of first path edge to the reported transaction
- reported amount versus traced amount ratio
(when reported amount is given)

13.3 Rule baseline (behavior/rules.py)

Implement documented rules, each producing one signal with a level:

- `rapid_pass_through`: onward transfer within a configured short interval at two or more consecutive hops.

- fan_out_splitting: one address forwards to many new addresses within an interval.
- fan_in_merging: many traced inputs merge to one address.
- peel_chain_pattern: repeated decreasing outputs with a large remainder forwarded.
- equal_value_repetition: repeated identical output amounts.
- new_address_chain: intermediate addresses with no history before the incident.
- asset_hopping: asset changes along the path. Thresholds live in configuration and are documented with rationale.

13.4 ML baseline (behavior/baseline_model.py)

- scikit-learn logistic regression and gradient boosting, trained only on a documented dataset (for example a public labelled dataset chosen after checking task fit, or synthetic data clearly marked).
- Features limited to Part 13.2. No identifiers, no addresses, no direct label leakage.
- Use time-based splits. Persist model with version, training data description, date, feature

list, and metrics in

backend/data/models/model_card.md.

- Return calibrated outputs only if calibration is evaluated (Platt or isotonic with reliability curve). Otherwise expose only a coarse pattern-similarity level, never a percentage.
- Provide feature-level explanations using coefficients or permutation importance summaries.
- If no trained model is present, the system runs rules only and states so in limitations.

13.5 Explanation output (behavior/explain.py)

For each signal, produce: display name, plain-language description, the supporting feature values, and the fixed limitation note: "This pattern can also result from legitimate activity."

PART 14. CROSS-REPORT CORROBORATION

14.1 Case subgraph representation

Each case stores its retained transfer set and extracted paths. Report metadata (narrative, contact details) is never used for matching.

14.2 Candidate generation

(corroboration/matcher.py)

For a new or updated case, compare against existing cases in the same chain:

- shared addresses among path nodes, excluding the reported address of either case unless it appears downstream in the other
- shared transaction hashes
- overlapping path segments (consecutive shared edges in the same direction)
- similar temporal structure (only as an inferred, lower-strength signal)

14.3 Exclusions and conservative thresholds

(corroboration/thresholds.py)

- Exclude nodes with an active label of a shared-infrastructure type or degree above `CORROBORATION_HUB_EXCLUSION_DEGREE`. If the only overlap consists of such nodes, record the link as weak with `shared_infrastructure_flag` true.
- Require temporal consistency: the time skew between the overlapping segments must be within `CORROBORATION_MAX_TIME_SKEW_DAYS`.

- Strength assignment follows Part 4.7.
Temporal-structure similarity alone can never exceed weak and is tagged inferred.

14.4 Output

Return for each link: the exact shared addresses, transactions, and segments, the strength, the evidence tag for each element, the shared infrastructure flag, temporal notes, and the mandatory caution text from the copy deck.

14.5 Temporal updating

- When new transfers are retrieved for an existing case or a new case is added, re-run matching for affected cases.
- Record `first_observed_at` and append to `change_history` with the specific new evidence that created or changed each link.
- Provide a manual refresh endpoint with rate limiting.

14.6 Privacy

Related cases are shown to a user only as case references and shared public-chain evidence. Do not reveal another reporter's narrative, contact details, or reported amounts. In public mode, related cases are visible only if the other case was

created with sharing consent; otherwise return only an aggregate count with the shared evidence, no reference.

PART 15. EVIDENCE AND CONFIDENCE ASSEMBLY

15.1 Assembler (evidence/assembler.py)

Combine outputs from graph, attribution, behavior, and corroboration into a single

CaseEvidencePackage:

- inputs and effective parameters
- data sources, retrieval times, and data snapshot id
- ranked paths with edges
- attributions with label provenance
- signals with explanations
- related cases
- limitations
- confidence_panel with four independent entries
- generated summary text

- versions: code, model, label registry snapshot hash

15.2 Headline state selection

Choose the headline in this order of precedence, deterministic and documented:

1. If retrieval failed entirely: status failed with a clear message.
2. If no transfers found for the address in the window: "no_paths_found".
3. If at least one path has verified_label_match: headline verified_label_match.
4. Else if at least one path has potential_association: headline potential_association.
5. Else if any path was truncated or broken before reaching a conclusion: insufficient_data.
6. Else: no_match_in_current_references.

15.3 Limitations generator

(evidence/limitations.py)

Always include, when applicable: adapter gaps, truncation, path breaks, label staleness, label scope, synthetic or demo data notice, unsupported chain or asset, partial analysis due

to limits, and the analysis timestamp. Include the standing statement: "This output is an investigative lead for human review and is not a legal finding."

15.4 Audit trail (evidence/audit.py)

Write an audit event for each pipeline stage with sanitized payloads and a SHA-256 hash of the payload. Provide a function to verify the audit chain integrity for a case. Never store secrets or raw narrative text in audit payloads.

PART 16. PDF EVIDENCE REPORT

Use ReportLab. Layout: A4 portrait, 20 mm margins, single sans-serif family for text, monospace for addresses and hashes. No color-coded verdicts. Neutral header and footer with case reference, page numbers, and analysis timestamp.

Sections in order:

1. Cover block: title, case reference, chain, reported address, analysis timestamp, report version, demonstration notice if applicable.

2. Statement of use: non-conclusive-use statement and definitions of evidence tags.
3. Summary: headline state and plain-language summary.
4. Confidence panel: four independent entries with definitions.
5. Input and parameters: all inputs, path-selection parameters, value-tracing method, limits.
6. Data sources: adapter, source names, retrieval times, truncation notes, snapshot id.
7. Fund movement: for each ranked path, a table of edges with hash, block, timestamp UTC, sender, receiver, asset, amount.
8. Potential exchange or service links: label details and provenance table, connection type, confidence factors.
9. Behavioral signals: signal, level, explanation, limitation.
10. Related reports: shared evidence tables, strength, caution text.
11. Limitations and data gaps.
12. Audit summary: event list with timestamps and payload hashes, code and model versions, label registry snapshot hash.

13. Glossary of terms.

Store the SHA-256 hash of the generated file in the reports table and print it in the footer of the final page.

Long addresses and hashes must wrap without truncation in the PDF.

PART 17. REST API SPECIFICATION

Base path /api/v1. JSON only. All responses include a request_id header. All errors use ErrorResponse.

17.1 POST /cases

Creates a case and starts analysis. For the prototype run the analysis in a background task and return 202 with case_reference and status pending. Provide polling through GET. In test mode, allow synchronous execution.

Validation errors return 422 with the specific field and code.

17.2 GET /cases/{case_reference}

Returns CaseSummaryResponse. While running, returns status running with progress stage names

(retrieval, graph, attribution, behavior, corroboration, assembly).

17.3 GET /cases/{case_reference}/paths

Returns ranked PathResponse list with pagination (limit, offset).

17.4 GET /cases/{case_reference}/attributions

Returns AttributionResponse list.

17.5 GET /cases/{case_reference}/signals

Returns SignalResponse list.

17.6 GET /cases/{case_reference}/related

Returns RelatedCaseResponse list respecting the privacy rules in 14.6.

17.7 POST /cases/{case_reference}/refresh

Re-runs retrieval and analysis, records what changed. Rate limited more strictly.

17.8 GET /cases/{case_reference}/report.pdf

Generates or returns the stored report. 409 with report_not_ready if analysis is incomplete.

17.9 GET /cases/{case_reference}/audit

Restricted to authorized roles. Returns the audit event list and integrity verification result.

17.10 GET /labels/{chain}/{address}

Public lookup returning active labels with

provenance for an address, or an explicit "no match in current references" response.

17.11 GET /health and GET /health/ready
Liveness, and readiness including database and adapter checks. No sensitive detail.

17.12 API conventions

- Version prefix, consistent snake_case fields, ISO 8601 UTC timestamps.
- Idempotency: identical create requests within a short window may return the existing case.
- Pagination on list endpoints.
- OpenAPI docs generated with descriptions and examples for every model.
- Security headers on all responses.

PART 18. FRONTEND SPECIFICATION

18.1 Pages and routes

- "/" HomePage: input form.
- "/cases/:caseReference" ResultPage: progress, then result.

- "*" NotFoundPage.

18.2 HomePage

Contents in order: product name and one-sentence description; the form; a short note stating what the tool does and does not establish; a link to the limitations page section.

Form:

- Wallet address (required, monospace input, paste-friendly, trims whitespace, live format check with inline message on blur, not on every keystroke).
- Chain selector (single option in MVP, still rendered for clarity).
- Disclosure titled "Additional details" containing transaction hash, incident date with precision selector, reported amount and asset.
- Primary button "Check Wallet". Disabled while submitting, shows progress text (no spinner-only state).
- Optional narrative field is not shown on the public form in MVP; supported by the API only.

18.3 ResultPage structure

Top bar: case reference, chain, analysis timestamp, export button, new check link.

Then sections in this order, each in a bordered panel with a clear heading:

1. Summary: headline status badge (neutral styling), plain-language paragraph, demonstration notice if applicable.
2. Confidence panel: four labelled rows (Path evidence, Attribution confidence, Behavioral signals, Corroboration) each with a text level and a one-line definition. No numeric score. No progress bars implying percentages.
3. Fund movement: ranked path list. Each path is a compact row (hop count, elapsed time, traced value with method note). Selecting a path opens the graph view and the edge table.
4. Potential exchange link: label entity, connection type, attribution state, label source and date, verification level, staleness note, scope note, shared infrastructure warning if flagged.
5. Why it was flagged: signal list with plain explanation and the limitation note.
6. Related reports: link cards showing strength, exact shared evidence, caution text.
7. Limitations: list with analysis timestamp and standing statement.

8. Export: button for PDF and a copy action for the case reference.

18.4 Components (frontend/src/components)

- form: AddressInput, OptionalDetails, SubmitButton
- result: SummaryPanel, ConfidencePanel, StatusBadge, LimitationsList, DemoNotice
- evidence: PathList, PathRow, EdgeTable, AttributionCard, LabelProvenanceTable, SignalList, RelatedCaseCard, EvidenceTagLabel
- graph: PathGraph (React Flow, left-to-right layout, nodes as address chips, edges labelled with date and amount, terminal labelled nodes outlined with the label name, break markers with reason)
- feedback: ProgressStages, ErrorPanel, EmptyState, PartialDataNotice, SkeletonBlock
- common: MonoText (truncated middle with copy control), CopyButton, ExpandablePanel, ExternalLink (with explorer link), Timestamp

18.5 States (each must be implemented and tested)

- initial and validating

- submitting
- running with named stages
- completed with all sections
- completed with no paths found
- completed with insufficient data
- partial due to limits or upstream issues
- failed with retry action
- rate limited with retry-after text
- not found
- offline or upstream unavailable

18.6 Behavior details

- Polling with backoff while the case is running, stop on terminal state.
- Deep links to result pages work on reload.
- Copy control on every address and hash; confirmation shown as text change, not an icon animation.
- Addresses show the first 6 and last 4 characters by default, full value on expand or in an accessible tooltip and in the edge table.
- All times displayed in UTC with an option to also show local time.

- Explorer links open in a new tab with rel noopener noreferrer.

18.7 Accessibility

- WCAG 2.1 AA.
- Semantic landmarks, one h1 per page, logical heading order.
- All controls reachable by keyboard, visible focus ring.
- Form errors linked with aria-describedby, announced with aria-live polite.
- Graph view has an equivalent table alternative that is always available.
- Color is never the only carrier of meaning.
- Respect prefers-reduced-motion. Use minimal motion.

PART 19. VISUAL DESIGN SYSTEM

19.1 Direction

Institutional, restrained, and document-like. The interface should resemble a well-made

government or research portal, not a consumer crypto app.

19.2 Strict prohibitions

- No emojis of any kind in code, UI, docs, or commit messages.
- No emoji-style or decorative icons. No typed arrows, checkmarks, stars, or ornamental symbols used as visual decoration.
- No gradient banners, glassmorphism, glow effects, neon colors, matrix or hacker themes, animated backgrounds, or particle effects.
- No red and green guilt verdicts, no traffic-light risk meters, no skull, shield-with-alert, or warning-siren imagery.
- No stock illustrations or three-dimensional coin imagery.

19.3 Design tokens

(frontend/src/styles/tokens.css)

Colors (light theme)

- background: #F7F8FA
- surface: #FFFFFF
- surface-subtle: #F1F3F6
- border: #D9DEE5

- border-strong: #B8C0CC
- text-primary: #1B2430
- text-secondary: #4A5565
- text-muted: #6B7686
- accent: #2F5D8A
- accent-hover: #264C72
- accent-subtle: #E6EEF6
- status-neutral: #4A5565 on #EDEFF3
- status-attention: #7A5A12 on #F6EFD9
(muted amber, used for caution and staleness)
- status-info: #2F5D8A on #E6EEF6
- focus-ring: #2F5D8A at 2px offset 2px
Verify all text and background pairs meet
WCAG AA contrast and adjust values if a pair
fails.
Provide a dark theme with equivalent tokens,
selected by prefers-color-scheme.

Typography

- Text: Inter or IBM Plex Sans, with system sans-serif fallback.
- Monospace: IBM Plex Mono or JetBrains Mono, used only for addresses, hashes, and case references.

- Scale: 12, 14, 16, 18, 20, 24, 30 px. Body 16 px, line height 1.5. Headings use weight 600, not color, to create hierarchy.
- Numeric data uses tabular figures.

Spacing and shape

- 4 px base unit. Use multiples of 4 and 8 only.
- Border radius 4 px for inputs and buttons, 6 px for panels.
- 1 px borders. Shadows limited to a single subtle elevation for popovers.
- Maximum content width 960 px, centered, generous vertical rhythm (32 px between panels).

19.4 Component styling rules

- Buttons: primary solid accent with white text; secondary outlined; no icon-only primary actions. Minimum target size 40 px height.
- Status badge: text label, neutral background, 1 px border, sentence case. Never colored to imply guilt.
- Panels: white surface, 1 px border, 24 px padding, heading at top with optional short definition beneath.

- Tables: compact, sticky header where long, zebra striping off, horizontal scroll on narrow screens with visible scroll affordance.
- Evidence tag labels: small text-only labels reading Observed, Derived, or Inferred, with a definition available in a tooltip and in the glossary.
- Icons: Lucide only, 16 or 20 px, stroke 1.5, used solely for copy, external link, expand and collapse, download, and close.

19.5 Responsive behavior

- Design mobile first from 360 px width.
- Path graph collapses to the table view on narrow screens with a toggle.
- Panels stack vertically. No horizontal scrolling of the page body.

PART 20. COPY DECK (APPROVED LANGUAGE)

Store in docs/copy-deck.md and frontend/src/lib/copy.ts. All user-facing strings come from copy.ts. No inline strings in components apart from labels bound to data.

20.1 Approved phrases

- "Potential exchange connection identified"
- "Verified label match"
- "Potential association"
- "No match found in current references"
- "Insufficient data"
- "No fund movement found for this address in the selected window"
- "Observed transfer"
- "Label source"
- "Connection type: direct, indirect, or inferred"
- "Pattern observed"

20.2 Prohibited phrases

"criminal exchange", "fraudulent exchange", "scam wallet confirmed", "guilty", "proof of fraud", "fraud score", "risk percentage", "definitely", "confirmed scammer", "the exchange is responsible", and any statement of intent or knowledge attributed to an exchange or wallet owner. Add a lint test that scans copy.ts, templates, and docs for these strings and fails the build if any appear.

20.3 Templates

Summary, potential_association:

"We checked address {address} on {chain} using

data retrieved on {date}. Funds sent from this address appear to have moved through {hop_count} transfer(s) to an address labelled {entity} by {label_source}. This is a potential connection based on observed transfers and a label of {verification_level} verification. It does not show that the service knew of or took part in any wrongdoing."

Summary, verified_label_match:

"We checked address {address} on {chain} using data retrieved on {date}. An observed transfer path reaches an address labelled {entity} in a source with {verification_level} verification. This describes where funds moved. It does not establish involvement or intent by the service."

Summary, no_match_in_current_references:

"We checked address {address} on {chain} using data retrieved on {date}. We traced observed transfers within the selected limits and did not reach an address present in our current label references. This does not rule out a connection to a service that is not in our references."

Summary, insufficient_data:

"We could not reach a conclusion for address

{address} on {chain}. {reason}. Please review the limitations section."

Summary, no_paths_found:

"No outgoing transfers were found for address {address} on {chain} in the selected time window."

Related report caution (mandatory on every related report):

"These cases share the on-chain evidence shown below. Shared addresses can result from common services, exchange deposit infrastructure, or coincidence, and do not by themselves indicate common ownership or coordination."

Signal limitation note (mandatory on every signal):

"This pattern can also result from legitimate activity."

Standing statement (report and result page):

"This output is an investigative lead for human review. It is not a legal finding and does not accuse any person or organization."

Demonstration notice:

"Demonstration mode. This result uses simulated data or synthetic labels and does not describe a real investigation."

PART 21. SECURITY AND PRIVACY

21.1 Input handling

- Validate and canonicalize every address and hash on the server. Reject on failure with specific codes.
- Enforce maximum body size and field lengths.
- Sanitize text fields; store narrative as data, render as escaped text only. Never render user text as HTML.

21.2 Secrets and configuration

- Secrets only in environment variables or a secret manager. Never in source control, logs, responses, or reports. Add a pre-commit secret scan.
- Frontend never calls the explorer API directly and never receives keys.

21.3 Transport and headers

- HTTPS in production. HSTS, Content-Security-Policy restricting sources, X-Content-Type-Options, Referrer-Policy no-referrer, X-Frame-Options deny.

- CORS allowlist from configuration.

21.4 Abuse protection

- Per-IP rate limits on case creation and refresh, with Retry-After header.
- Concurrency cap on active analyses. Queue with a maximum length, return a clear busy response when full.
- Request timeouts and cancellation of abandoned analyses.

21.5 Access control

- Public endpoints: create case, read own case by unguessable reference, label lookup, health.
- Case references used in public URLs contain a random component of at least 128 bits (for example TX-2026-000123 as the display reference plus a separate access token) so that enumeration is not possible.
- Roles for non-public data: investigator (audit trail, private details with consent), administrator (label management). Implement with a simple token-based scheme for the prototype and document the intended production identity integration.

21.6 Privacy

- Collect the minimum. No accounts for public users in the MVP.
- Private details encrypted at the application level with a key from configuration.
- Retention: automated deletion job removes expired cases, private details, and reports per configured retention days, writing a `case_deleted` audit event.
- Do not attempt to identify private individuals from addresses. Do not add features that link addresses to personal identities.
- Logs exclude narrative text and contact details. Log addresses only at debug level in development.

21.7 Dependency and supply chain

- Pin dependencies. Run `pip-audit` and `npm audit` in CI and fail on high severity.
- Use a minimal container base image and run as a non-root user.

PART 22. TESTING STRATEGY

22.1 Backend unit tests

- Address validation: valid checksum, invalid checksum, wrong length, wrong chain, mixed case, whitespace.
- Adapter normalization: decimals handling, failed transactions, pagination, truncation warning, empty results, upstream error mapping (use respx).
- Graph: seed selection with and without tx hash, time-respecting paths, budget enforcement, hub exclusion, terminal labelled nodes, deduplication.
- Value tracing: proportional and fifo on hand-computed examples, asset change breaks, continuity ratio bounds.
- Attribution: every state in Part 4.3, every confidence rule, staleness, scope caveat, synthetic handling in and out of demo mode.
- Behavior: each rule with positive and negative fixtures, threshold boundaries.
- Corroboration: strong, moderate, weak, and none cases; shared infrastructure exclusion; time skew limit; privacy filtering.
- Evidence: headline precedence order, limitations generation, audit hash chain

verification.

- Reports: PDF generated, contains required sections and text, hash stored, long hashes wrap.

22.2 Property-based tests (hypothesis)

- No returned path has a decreasing timestamp.
- Node and edge budgets are never exceeded.
- Traced value never exceeds the originating transfer value for the asset.
- Output is identical across repeated runs with the same inputs and fixtures.

22.3 Integration tests

- Full case creation through report download on the mock adapter for each fixture scenario.
- Upstream timeout produces a partial or failed case with a clear limitation.
- Rate limiting returns 429 with Retry-After.
- Retention job deletes expired data.

22.4 Frontend tests

- Component tests for each state in 18.5.
- Form validation messages and disabled states.

- Copy lint test for prohibited phrases.
- Accessibility checks with axe on Home and Result pages.
- One Playwright end-to-end run: enter demonstration address, view result, expand evidence, download report.

22.5 Fixture scenarios (backend/data/fixtures)

Each fixture has a JSON description of addresses, transfers, labels, and expected results.

1. `direct_exchange_deposit`: reported address sends directly to a labelled exchange address.
2. `multi_hop_indirect`: three intermediaries before a labelled address.
3. `split_and_merge`: value splits into several addresses and merges before the terminal address.
4. `mixer_break`: path enters a labelled mixer, must end with a `mixer_interaction` break and `insufficient_data`.
5. `bridge_break`: path enters a bridge, must show `bridge_interaction` break.
6. `swap_asset_change`: value swaps to another token, must show `swap_asset_change` break.

7. unlabelled_destination: path ends at an unknown address, must yield no_match_in_current_references.
 8. stale_label: label older than the staleness threshold, must lower confidence and show a staleness note.
 9. shared_infrastructure_overlap: two cases overlap only on an exchange hot wallet, must be weak with the flag and must not claim common ownership.
 10. strong_corroboration: two cases share a two-edge downstream segment through non-infrastructure addresses.
 11. truncated_history: address history exceeds the fetch cap, must show truncation limitation.
 12. no_outgoing_transfers: must yield no_paths_found.
 13. malformed_input_set: invalid addresses, hashes, dates, future dates.
 14. upstream_failure: simulated timeout and rate limit from the explorer.
-

PART 23. EVALUATION PLAN

Implement in backend/app/evaluation and document in docs/evaluation.md. Design experiments and success criteria before tuning thresholds. Record all results in machine-readable files under docs/evaluation/results and summarize in the document.

23.1 Data

- Public labelled research data (for example Elliptic and Elliptic++ style datasets) for the wallet and transaction classification baseline, only where the chain and task match. State dataset version, license, class balance, and label semantics.
- Synthetic case generator producing victim-report-like cases with controllable noise (missing hash, approximate date, wrong amount), clearly marked synthetic.
- Curated label set with verified sources for attribution accuracy tests.

23.2 Experiments

1. Wallet behavior detection: rule baseline and standard ML classifier versus the interpretable behavioral model. Metrics: precision, recall, F1, PR-AUC, false-positive rate.

2. Path relevance: unfiltered bounded traversal versus report-conditioned prioritization.
Metrics: relevant paths surfaced at k, irrelevant path volume, runtime.
3. Entity attribution: single-label lookup versus provenance-aware confidence output. Metrics: accuracy on verified labels, expected calibration error where applicable, abstention rate, unknown handling.
4. Report corroboration: independent evaluation versus cross-report matching. Metrics: pairwise precision and recall, false association count (including shared infrastructure traps), evidence traceability rate.
5. Temporal robustness: random split versus time-based split. Metrics: drift in recall and false-positive rate over time windows.
6. Noisy inputs: complete cases versus missing hash, approximate date, incomplete report.
Metrics: robustness, abstention rate, failure clarity.
7. Usability: five to ten participants complete a task with the plain-language screen versus a technical output. Metrics: task completion, comprehension quiz score, time to result.

Document participant consent and anonymization.

23.3 Ablations

- Remove report conditioning.
- Remove attribution provenance and confidence.
- Disable cross-report corroboration.
- Remove temporal features.
- Compare interpretable ML against any optional graph model only if one is built.

23.4 Rigor rules

- Use time-based splits where possible. Prevent leakage across graph neighborhoods and closely related transactions.
- Report imbalance, label sources, missing labels, and chain-specific limits.
- Set a fixed random seed and record library versions.
- Report failure cases as well as successes.
- Do not present research dataset results as real-world performance on unverified victim reports.

- System measures: latency percentiles, evidence completeness rate, cost per analysis (upstream calls), correct abstention rate.

23.5 Output

Provide a single command "make eval" producing a results directory and a generated evaluation summary with tables. Include an honest limitations paragraph.

PART 24. LOGGING AND OBSERVABILITY

- Structured JSON logs with request_id, case_reference, stage, duration_ms, outcome.
 - Metrics counters for cases created, cases by headline state, upstream calls, cache hits, failures, and latency histograms.
 - No sensitive data in logs (see 21.6).
 - A single correlation id passes from request through background analysis to audit events.
-

PART 25. DEVELOPMENT ENVIRONMENT AND CI

- docker-compose starts db, api, and frontend with hot reload. Demo mode is the default in local compose, with the mock adapter.
 - Makefile targets as listed in 5.3, each documented in README.
 - CI pipeline stages: install, lint, type check, backend tests with coverage, frontend tests, build, dependency audit, copy lint, secret scan.
 - README includes: overview, safety and limitations summary, quick start, configuration table, running tests, running evaluation, project structure, contribution notes. README must not overstate capability and must include the "Claims to avoid" list from Part 27.
-

PART 26. PHASED BUILD PLAN WITH ACCEPTANCE CRITERIA

Phase 1. Scaffold and foundations

Deliver: repository structure, configuration loader, logging, error model, health endpoints, Docker and

Makefile, CI, mock adapter with two fixtures,
frontend shell with design tokens.

Acceptance: make dev starts all services; health
endpoints respond; CI is green; copy lint runs;
design tokens render a sample page that passes
contrast checks.

Phase 2. Validation, retrieval, storage, basic result
page

Deliver: address, hash, and date validation;
adapter interface and Etherscan-compatible
adapter; normalization; database models and
migrations; POST and GET case endpoints;
HomePage and a basic ResultPage listing
normalized transfers.

Acceptance: malformed input yields specified
error codes; a fixture address returns normalized
transfers; adapter unit tests including truncation
and error mapping pass.

Phase 3. Graph tracing

Deliver: bounded expansion, path extraction, value
tracing, breaks, ranking, pattern detection, path
API, PathList and PathGraph with table alternative.

Acceptance: fixtures 1 to 6 and 11 to 12 produce
expected paths and breaks; property tests pass;
budgets and deadlines respected.

Phase 4. Attribution and provenance

Deliver: label registry, seed loader, admin CLI, matcher, confidence rules, shared infrastructure list, attribution API and UI card with provenance table.

Acceptance: every attribution state reachable in tests; fixtures 7 and 8 behave as specified; synthetic labels rejected outside demo mode.

Phase 5. Behavioral signals

Deliver: feature extraction, rules, ML baseline with model card (if suitable data is available), explanations, signals API and UI.

Acceptance: each rule has positive and negative tests; signals always carry limitation notes; no numeric fraud score appears anywhere.

Phase 6. Cross-report corroboration

Deliver: subgraph storage, matcher, thresholds, privacy filtering, temporal updating, related API and UI.

Acceptance: fixtures 9 and 10 behave as specified; shared infrastructure never yields more than weak; related cases never expose private details.

Phase 7. Evidence, audit, report, polish

Deliver: assembler, headline precedence,

limitations generator, audit hash chain, PDF report, full ResultPage with all states, accessibility pass, retention job, security headers and rate limits. Acceptance: PDF contains all sections; audit verification passes; axe checks pass; all state tests pass; security checklist in docs/security-and-privacy.md completed.

Phase 8. Evaluation, documentation, demo Deliver: evaluation scripts and results, all docs completed, demo script, final README, known issues list.

Acceptance: make eval reproduces results; docs match behavior; demo script runs end to end on the mock adapter.

PART 27. CLAIMS TO AVOID AND DEFINITION OF DONE

27.1 Claims to avoid in code comments, docs, UI, README, and presentation material

- Detection of every fraudulent exchange.
- Any statement that a transfer to an exchange proves the exchange is involved.
- Definitive fraud verdicts or AI-determined guilt.

- That no existing platform provides graph tracing or exchange attribution.
- High accuracy before suitable metrics are measured and reported.
- Complete traceability across mixers, bridges, or privacy systems.

27.2 Novelty statement (use exactly this framing)

"The contribution is an integrated, victim-report-conditioned, uncertainty-aware investigation workflow combining case-focused tracing, provenance-aware exchange attribution, and cross-report corroboration in one explainable output. Novelty is a hypothesis to be validated by literature review, baseline comparison, and ablation."

27.3 Definition of done for the prototype

- All eight phases meet acceptance criteria.
- Test suites pass; coverage target met; type checks and lint pass.
- No emojis or decorative symbols anywhere in the repository (add a CI check scanning for non-ASCII pictographic characters).
- No prohibited phrases anywhere (copy lint passes).

- Every result view and report includes limitations and the standing statement.
 - Demo mode notice appears whenever synthetic data or labels are used.
 - Documentation is complete, accurate, and consistent with behavior.
-

PART 28. DEMONSTRATION SCRIPT

Provide in docs/demo-script.md, runnable on the mock adapter in demo mode.

1. Open the home page and state the tool purpose and limits.
2. Enter the labelled demonstration address and select "Check Wallet".
3. Show the running stages, then the result summary with the demonstration notice.
4. Open the confidence panel and explain that four independent entries are shown instead of one score.
5. Open the top-ranked path, show the table and graph, and point out transaction hashes, timestamps, and amounts.

6. Open the potential exchange link and show label source, verification level, staleness, and scope notes.
 7. Open behavioral signals and read one limitation note.
 8. Enter a second demonstration address that shares a downstream segment and show the related report with the exact overlap and the caution text.
 9. Show the shared-infrastructure fixture to demonstrate that overlap on a hot wallet is flagged and not treated as common ownership.
 10. Export the PDF and walk through the audit summary and limitations.
 11. Show the insufficient data case (mixer break) to demonstrate abstention.
-

PART 29. FIRST ACTIONS

Begin now. Respond with:

1. A restatement of the plan for Phase 1 as a numbered list.
2. Any assumptions you are making.

3. The list of files you will create.

Then implement Phase 1 completely, run all checks, and report results before asking to continue to Phase 2.